Version: 11.x

Set up with React Server Components

This guide is an overview of how one may use tRPC with a React Server Components (RSC) framework such as Next.js App Router. Be aware that RSC on its own solves a lot of the same problems tRPC was designed to solve, so you may not need tRPC at all.

There are also not a one-size-fits-all way to integrate tRPC with RSCs, so see this guide as a starting point and adjust it to your needs and preferences.

> **INFO**
> If you're looking for how to use tRPC with Server Actions, check out this blog post by Julius.

> **CAUTION**
> Please read React Query's Advanced Server Rendering docs before proceeding to understand the different types of server rendering and what footguns to avoid.

Add tRPC to existing projects

1. Install deps

npm yarn pnpm bun deno

```
npm install @trpc/server @trpc/client @trpc/tanstack-react-query @tans
```

2. Create a tRPC router

Initialize your tRPC backend in `trpc/init.ts` using the `initTRPC` function, and create your first router. We're going to make a simple "hello world" router and procedure here - but for deeper information on creating your tRPC API you should refer to the Quickstart guide and Backend usage docs for tRPC information.

> **INFO**
> The file names used here are not enforced by tRPC. You may use any file structure you wish.

▸ View sample backend

3. Create a Query Client factory

Create a shared file `trpc/query-client.ts` that exports a function that creates a `QueryClient` instance.

trpc/query-client.ts

```ts
import {
  defaultShouldDehydrateQuery,
  QueryClient,
} from '@tanstack/react-query';
import superjson from 'superjson';

export function makeQueryClient() {
  return new QueryClient({
    defaultOptions: {
      queries: {
        staleTime: 30 * 1000,
      },
      dehydrate: {
        // serializeData: superjson.serialize,
        shouldDehydrateQuery: (query) =>
          defaultShouldDehydrateQuery(query) ||
          query.state.status === 'pending',
      },
      hydrate: {
        // deserializeData: superjson.deserialize,
      },
    },
  });
}
```

We're setting a few default options here:

- `staleTime`: With SSR, we usually want to set some default staleTime above 0 to avoid refetching immediately on the client.
- `shouldDehydrateQuery`: This is a function that determines whether a query should be dehydrated or not. Since the RSC transport protocol supports hydrating promises over the network, we extend the `defaultShouldDehydrateQuery` function to also include queries that are still pending. This will allow us to start prefetching in a server component high up the tree, then consuming that promise in a client component further down.
- `serializeData` and `deserializeData` (optional): If you set up a data transformer in the previous step, set this option to make sure the data is serialized correctly when hydrating the query client over the server-client boundary.

4. Create a tRPC client for Client Components

The `trpc/client.tsx` is the entrypoint when consuming your tRPC API from client components. In here, import the **type definition** of your tRPC router and create typesafe hooks using `createTRPCContext`. We'll also export our context provider from this file.

trpc/client.tsx

```tsx
'use client';

// ^-- to make sure we can mount the Provider from a server component
import type { QueryClient } from '@tanstack/react-query';
import { QueryClientProvider } from '@tanstack/react-query';
import { createTRPCClient, httpBatchLink } from '@trpc/client';
import { createTRPCContext } from '@trpc/tanstack-react-query';
import { useState } from 'react';
import { makeQueryClient } from './query-client';
import type { AppRouter } from './routers/_app';

export const { TRPCProvider, useTRPC } = createTRPCContext<AppRouter>();

let browserQueryClient: QueryClient;
function getQueryClient() {
  if (typeof window === 'undefined') {
    // Server: always make a new query client
    return makeQueryClient();
  }
  // Browser: make a new query client if we don't already have one
  // This is very important, so we don't re-make a new client if React
  // suspends during the initial render. This may not be needed if we
  // have a suspense boundary BELOW the creation of the query client
  if (!browserQueryClient) browserQueryClient = makeQueryClient();
  return browserQueryClient;
}

function getUrl() {
  const base = (() => {
    if (typeof window !== 'undefined') return '';
    if (process.env.VERCEL_URL) return `https://${process.env.VERCEL_URL}`;
    return 'http://localhost:3000';
  })();
  return `${base}/api/trpc`;
}

export function TRPCReactProvider(
  props: Readonly<{
    children: React.ReactNode;
  }>,
) {
  // NOTE: Avoid useState when initializing the query client if you don't
  //       have a suspense boundary between this and the code that may
  //       suspend because React will throw away the client on the initial
  //       render if it suspends and there is no boundary
  const queryClient = getQueryClient();

  const [trpcClient] = useState(() =>
    createTRPCClient<AppRouter>({
      links: [
        httpBatchLink({
          // transformer: superjson, <-- if you use a data transformer
          url: getUrl(),
        }),
      ],
    }),
  );
```

```
  ),

  return (
    <QueryClientProvider client={queryClient}>
      <TRPCProvider trpcClient={trpcClient} queryClient={queryClient}>
        {props.children}
      </TRPCProvider>
    </QueryClientProvider>
  );
}
```

Mount the provider in the root of your application (e.g. `app/layout.tsx` when using Next.js).

5. Create a tRPC caller for Server Components

To prefetch queries from server components, we create a proxy from our router. You can also pass in a client if your router is on a separate server.

trpc/server.tsx

```
import 'server-only'; // <-- ensure this file cannot be imported from the client

import { createTRPCOptionsProxy } from '@trpc/tanstack-react-query';
import { cache } from 'react';
import { createTRPCContext } from './init';
import { makeQueryClient } from './query-client';
import { appRouter } from './routers/_app';

// IMPORTANT: Create a stable getter for the query client that
//            will return the same client during the same request.
export const getQueryClient = cache(makeQueryClient);

export const trpc = createTRPCOptionsProxy({
  ctx: createTRPCContext,
  router: appRouter,
  queryClient: getQueryClient,
});

// If your router is on a separate server, pass a client:
createTRPCOptionsProxy({
  client: createTRPCClient({
    links: [httpLink({ url: '...' })],
  }),
  queryClient: getQueryClient,
});
```

Using your API

Now you can use your tRPC API in your app. While you can use the React Query hooks in client components just like you would in any other React app, we can take advantage of the RSC capabilities by prefetching queries in a server component high up the tree. You may be familiar with this concept as "render as you fetch" commonly implemented as loaders. This means the request fires as soon as possible but without suspending until the data is needed by using the `useQuery` or `useSuspenseQuery` hooks.

This approach leverages Next.js App Router's streaming capabilities, initiating the query on the server and streaming data to the client as it becomes available. It optimizes both the time to first byte in the browser and the data fetch time, resulting in faster page loads. However, `greeting.data` may initially be `undefined` before the data streams in.

If you prefer to avoid this initial undefined state, you can `await` the `prefetchQuery` call. This ensures the query on the client always has data on first render, but it comes with a tradeoff - the page will load more slowly since the server must complete the query before sending HTML to the client.

app/page.tsx

```tsx
import { dehydrate, HydrationBoundary } from '@tanstack/react-query';
import { getQueryClient, trpc } from '~/trpc/server';
import { ClientGreeting } from './client-greeting';

export default async function Home() {
  const queryClient = getQueryClient();
  void queryClient.prefetchQuery(
    trpc.hello.queryOptions({
      /** input */
    }),
  );

  return (
    <HydrationBoundary state={dehydrate(queryClient)}>
      <div>...</div>
      {/** ... */}
      <ClientGreeting />
    </HydrationBoundary>
  );
}
```

app/client-greeting.tsx

```tsx
'use client';

// <-- hooks can only be used in client components
import { useQuery } from '@tanstack/react-query';
import { useTRPC } from '~/trpc/client';

export function ClientGreeting() {
  const trpc = useTRPC();
  const greeting = useQuery(trpc.hello.queryOptions({ text: 'world' }));
  if (!greeting.data) return <div>Loading...</div>;
  return <div>{greeting.data.greeting}</div>;
}
```

TIP

You can also create a `prefetch` and `HydrateClient` helper functions to make it a bit more consice and reusable:

trpc/server.tsx

```tsx
export function HydrateClient(props: { children: React.ReactNode }) {
  const queryClient = getQueryClient();
  return (
    <HydrationBoundary state={dehydrate(queryClient)}>
      {props.children}
    </HydrationBoundary>
  );
}

export function prefetch<T extends ReturnType<TRPCQueryOptions<any>>>(
  queryOptions: T,
) {
  const queryClient = getQueryClient();
  if (queryOptions.queryKey[1]?.type === 'infinite') {
    void queryClient.prefetchInfiniteQuery(queryOptions as any);
  } else {
    void queryClient.prefetchQuery(queryOptions);
  }
}
```

Then you can use it like this:

```
import { HydrateClient, prefetch, trpc } from '~/trpc/server';

function Home() {
  prefetch(
    trpc.hello.queryOptions({
      /** input */
    }),
  );

  return (
    <HydrateClient>
      <div>...</div>
      {/** ... */}
      <ClientGreeting />
    </HydrateClient>
  );
}
```

Leveraging Suspense

You may prefer handling loading and error states using Suspense and Error Boundaries. You can do this by using the `useSuspenseQuery` hook.

app/page.tsx

```
import { HydrateClient, prefetch, trpc } from '~/trpc/server';
import { Suspense } from 'react';
import { ErrorBoundary } from 'react-error-boundary';
import { ClientGreeting } from './client-greeting';

export default async function Home() {
  prefetch(trpc.hello.queryOptions());

  return (
    <HydrateClient>
      <div>...</div>
      {/** ... */}
      <ErrorBoundary fallback={<div>Something went wrong</div>}>
        <Suspense fallback={<div>Loading...</div>}>
          <ClientGreeting />
        </Suspense>
      </ErrorBoundary>
    </HydrateClient>
  );
}
```

app/client-greeting.tsx

```tsx
'use client';

import { useSuspenseQuery } from '@tanstack/react-query';
import { trpc } from '~/trpc/client';

export function ClientGreeting() {
  const trpc = useTRPC();
  const { data } = useSuspenseQuery(trpc.hello.queryOptions());
  return <div>{data.greeting}</div>;
}
```

Getting data in a server component

If you need access to the data in a server component, we recommend creating a server caller and using it directly. Please note that this method is detached from your query client and does not store the data in the cache. This means that you cannot use the data in a server component and expect it to be available in the client. This is intentional and explained in more detail in the Advanced Server Rendering guide.

trpc/server.tsx

```tsx
// ...
export const caller = appRouter.createCaller(createTRPCContext);
```

app/page.tsx

```tsx
import { caller } from '~/trpc/server';

export default async function Home() {
  const greeting = await caller.hello();
  //    ^? { greeting: string }

  return <div>{greeting.greeting}</div>;
}
```

If you **really** need to use the data both on the server as well as inside client components and understand the tradeoffs explained in the Advanced Server Rendering guide, you can use `fetchQuery` instead of `prefetch` to have the data both on the server as well as hydrating it down to the client:

app/page.tsx

```tsx
import { getQueryClient, HydrateClient, trpc } from '~/trpc/server';

export default async function Home() {
  const queryClient = getQueryClient();
  const greeting = await queryClient.fetchQuery(trpc.hello.queryOptions());

  // Do something with greeting on the server

  return (
    <HydrateClient>
      <div>...</div>
      {/** ... */}
      <ClientGreeting />
    </HydrateClient>
  );
}
```